ToolAgent: A Small-Scale, DFSDT-Inspired Tool-Use Agent Built After ToolLLM

Ayush Mathur
Student ID: 202411013

Rudra Mandloi
Student ID: 202411082

Yuvank Goyal
Student ID: 202411100

Department of Computer Science / Information Technology
Academic Project, Phase 1, 2026

***Abstract*—ToolLLM showed how a language model can be taught to pick and call real web APIs, and how a tree search over reasoning steps (DFSDT) helps on harder multi-tool tasks [1]. In Phase 1 we rebuilt the main pieces at a size a student team can run on a laptop: a registry of 11 tool wrappers (3 offline mocks and 8 public web APIs), a retriever with TF-IDF, sentence-embedding and keyword back ends, a ReAct agent, a DFSDT-inspired agent with depth 4 and breadth 3, an LLM-as-judge evaluator, and a Streamlit interface. On our own 19-instruction benchmark the ReAct agent scored 89% and the DFSDT-inspired agent 84%. These numbers are provisional: reading the results showed that in 37 of 38 runs the model wrote the whole episode, including invented tool outputs, in a single reply, and our loop accepted it. We describe this bug, three argument-parsing bugs and a retrieval weakness that we found, and what still has to be done before the numbers can be trusted.**

***Index Terms*—tool use, language model agents, ReAct, depth-first search, API retrieval**

I. Introduction

Our project, which we call ToolAgent, asks a simple question: if a language model can call tools, what does it take to make it pick the right tool, call it with the right arguments, and give an answer based on what the tool actually returned? The ToolLLM paper answers this at a very large scale [1]. We cannot repeat that work, so we built a small version of each part and wired them together ourselves.

The code is split into four Python packages: `apis/` (tools), `retriever/` (choosing candidate tools), `agent/` (the reasoning loops and model back ends) and `eval/` (judging and reports). A Streamlit app (`app.py`) and three scripts sit on top. In total the repository holds about 3,170 lines of Python, of which 689 are the UI and 622 are tests.

Our contributions in Phase 1 are:

- a uniform `BaseAPI` interface and 11 tool wrappers;
- three interchangeable retrievers and a small measurement of how they rank tools on our benchmark;
- a ReAct loop and a DFSDT-inspired search loop that share one prompt builder and one action parser;
- a benchmark harness with an LLM judge, plus 52 unit tests;
- an honest account of the bugs that make our first benchmark numbers unreliable.

What we did not do. We did not use the ToolBench dataset, we did not fine-tune any model (there is no ToolLLaMA here), and our tool set has 11 wrappers instead of the 16,000+ RapidAPI endpoints in ToolLLM [1]. Our search agent borrows the idea of DFSDT but is not an exact reproduction of it (Section IV-D).

II. Background

A. ToolLLM

ToolLLM [1] collects a large set of real REST APIs from RapidAPI, uses a strong model to write instructions and solution paths for them (the ToolBench dataset), and fine-tunes LLaMA on that data to get ToolLLaMA. It adds an API retriever so the model only sees a few relevant APIs per query, and an automatic evaluator, ToolEval. We copy the shape of this pipeline (retrieve, reason, evaluate) but none of its data or models.

B. ReAct

ReAct [2] has the model alternate between a short piece of free-text reasoning and an action, and then feeds the result of the action back before the next step. The loop ends when the model decides it can answer. Our `ReActAgent` uses this Thought / Action / Observation format.

C. DFSDT and tree search

ReAct follows one chain, so one bad action can sink a whole run. Tree of Thoughts [3] treats reasoning as a search over a tree of partial solutions with breadth-first or depth-first strategies. ToolLLM applies the depth-first variant to tool calling and calls it DFSDT: the agent explores one branch deeply, and when that branch fails it goes back and tries a different next step [1].

D. Retrieval

With many tools, the prompt cannot list them all, so a retriever picks a short list. We tried three scoring methods. TF-IDF [4] (via scikit-learn [5]) weights shared words by how rare they are. Jaccard overlap [6] just counts shared words. Sentence-BERT [7] maps whole sentences to vectors

so that texts with similar meaning are close even when they share no words; we use the small `all-MiniLM-L6-v2` model, which is based on MiniLM [8].

E. Evaluation

ToolLLM reports a pass rate (did the run solve the task) and a win rate (which of two runs is better), both judged by a model [1]. Using a strong model as a judge is common, but such judges have known biases, including favouring answers written by themselves and favouring longer answers [9]. This matters for us, as Section VIII explains.

III. System Design and Architecture

Fig. 1 shows how the parts connect. A user instruction comes in through the Streamlit app or a script. The retriever scores every tool in the registry against the instruction and passes the top-k tools to an agent. The agent builds a prompt, asks a `BaseLLM` for text, parses an action from that text, runs the tool and appends the observation to the history. When the agent returns a final answer, the benchmark harness sends it to the `LLMJudge` and writes the verdicts to `results/`.

We kept three boundaries on purpose. Any `BaseAPI` can be registered without touching the retriever or agents. Any `BaseLLM` can drive either agent, which let us add Gemini later in one commit and lets the tests run with no network. Any list of rows can be scored by the `eval/` code. These boundaries paid off when we switched model providers in the middle of the project.

IV. Implementation

A. Tool registry

Every tool subclasses `BaseAPI` and declares a `name`, a one-line `description`, a `parameters` dictionary and a `call()` method that returns an `APIResponse(success, data, error)`. Tools report errors in this object instead of raising, so the agent can show a failure to the model as an observation.

There are three offline tools. `calculator` parses the expression with Python's `ast` module and only allows numbers and the operators `+ - * / ** %` (and unary signs), so a model cannot run arbitrary code through it. `weather` is a hard-coded table for Paris, Tokyo and New York, and `search` matches words against four fixed sentences. The online wrappers call Frankfurter for currency [10], the Wikipedia summary endpoint [11], the Free Dictionary API [12], JokeAPI [13], Quotable [14], REST Countries [15], OpenWeatherMap and NewsAPI.

`apis/registry.py` builds three registries: `default_registry()` with the three mocks, `full_registry()` with all 11, and `keyless_registry()` with the 9 tools that need no key of their own. We added the keyless one so that the benchmark only needs a model key.

B. Retriever

`APIRetriever` joins each tool's name and description into one string, fits an embedder on those strings, and ranks tools by cosine similarity to the instruction. The embedder only needs `fit()` and `similarity()`, so `TfidfEmbedder` and `SentenceTransformerEmbedder` are interchangeable. `KeywordMatchRetriever` is a separate baseline that scores by Jaccard overlap of lower-cased word sets. TF-IDF is the default because it needs no model download.

C. ReAct agent

`ReActAgent.run()` loops up to `max_steps` = 6 times. Each turn, `build_prompt()` concatenates the instructions in `REACT_SYSTEM_PROMPT`, the tool list, the user instruction and the history so far, and sends it as one user message. The reply is checked first for `Finish[...]` and then for `Action: name[args]`. If an action is found, `parse_args()` turns the bracket text into keyword arguments, the tool is called, and `Observation: ...` is added to the history. Unknown tool names and exceptions become `Error:` observations instead of crashing the run.

D. DFSDT-inspired agent

`DFSDTAgent` reuses the same prompt and parser but replaces the loop with a recursive `_search()` (Fig. 2). At each depth it asks the model for up to `branching_factor` = 3 replies. A reply that finishes ends the search. A reply with an unparseable action, an unknown tool, an exception or a failed `APIResponse` is dropped and the next sibling is sampled. A reply with a successful call goes one level deeper with the new observation in the history. If the whole subtree returns nothing, the loop moves to the next sibling, which is the backtracking step. Depth is capped at 4. The counter `explored_paths` records how many replies were sampled.

We call this "DFSDT-inspired" because it differs from ToolLLM's version in three ways. First, siblings are drawn by sending the exact same prompt again, so variety comes only from sampling randomness; the failed sibling is not shown to the model. ToolLLM's version, as we understand it, makes the model aware of earlier attempts so it tries something new [1]. Second, our search has no explicit "give up and restart" action. Third, a `Finish` reply is accepted without any check that it is supported by observations.

E. LLM layer

All model access goes through `BaseLLM`, whose only method is `generate(prompt)`. Two classes wrap the vendor SDKs (`AnthropicLLM` and `GeminiLLM`), and `EchoLLM` replays a scripted list of replies for tests. The benchmark and UI pick the back end from the `BENCHMARK_PROVIDER` variable in a local `.env` file. The default is Gemini with `gemini-3.5-flash-lite` [16]. Our `.env` sets only the provider, so the benchmark ran with these defaults. `GeminiLLM` retries on HTTP 429, 500, 502, 503 and 504

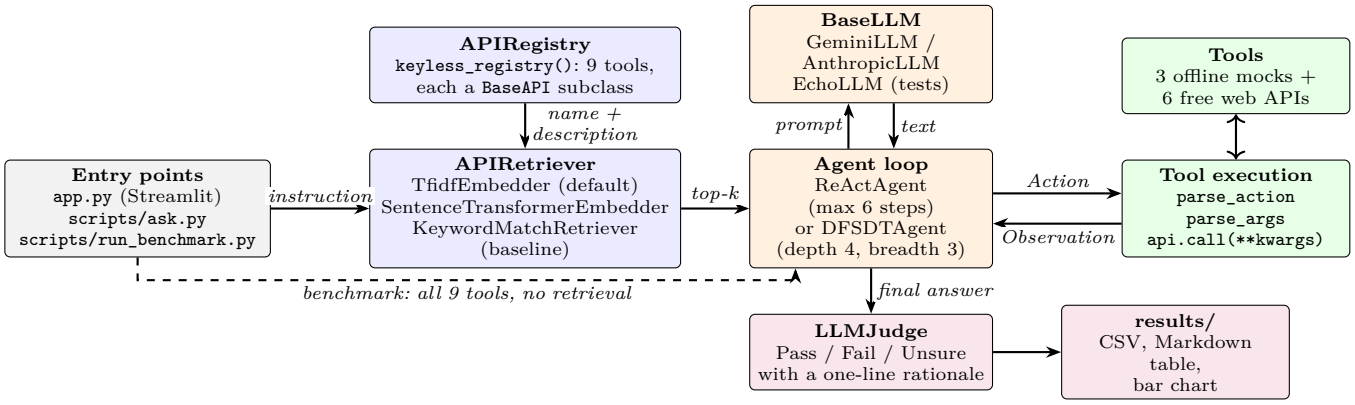


Fig. 1. Architecture of ToolAgent. Boxes name the real classes and files. The dashed path marks a detail that is easy to miss: the benchmark script hands all nine keyless tools straight to the agents, so retrieval quality does not affect the benchmark numbers.

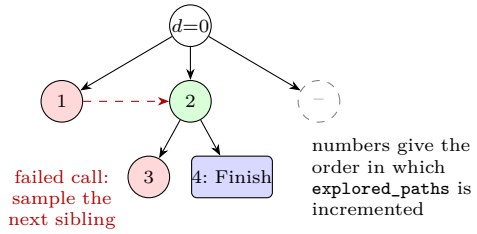


Fig. 2. Order of exploration in `DFSDTAgent._search`. Node 1's tool call fails, so a sibling is sampled; node 2 succeeds and the search goes deeper. The third root branch is never sampled.

with exponential backoff that starts at 2 s and doubles, for up to 5 retries.

F. Streamlit UI

`app.py` has three tabs: *Run Agent*, *Compare ReAct vs DFSDT* and *Results Dashboard*. A sidebar selects the strategy and the number of tools to retrieve (1 to 9, default 5). Unlike the benchmark, the UI does use the TF-IDF retriever. Each run shows the candidate tools, the final answer and a step-by-step trace that splits every reply into its thought, action and observation, plus a badge when DFSDT explored more branches than it kept. Registry and retriever are cached with `st.cache_resource`, and `friendly_error()` turns rate-limit, timeout and missing-key errors into short messages. The UI is built with Streamlit [17].

V. Evaluation Method and Results

A. Benchmark and procedure

`data/benchmark_instructions.json` holds 19 instructions that we wrote: 9 *easy* (one tool), 6 *medium* (two tools) and 4 *hard* (three tools). The script `scripts/run_benchmark.py` runs both agents on each one with all 9 keyless tools as candidates and sends each final answer to `LLMJudge.judge_pass()`, which replies *Pass*, *Fail* or *Unsure* with a one-sentence rationale. Only *Pass* counts toward the pass rate. The judge uses the same model as the agents. The script writes `benchmark_results.csv`, `comparison_table.md` and a bar chart. A win-rate comparator (`LLMJudge.compare()`) is implemented and unit-tested but was not part of this run.

TABLE I
PROVISIONAL PASS RATES FROM `RESULTS/COMPARISON_TABLE.MD`

Complexity	ReAct	DFSDT-insp.	N
Easy	89% (8/9)	89% (8/9)	9
Medium	83% (5/6)	67% (4/6)	6
Hard	100% (4/4)	100% (4/4)	4
Overall	89% (17/19)	84% (16/19)	19
Win rate, DFSDT vs ReAct: [**TO BE FILLED**]			

Status note. All benchmark numbers in this section are provisional. They come from the run committed on 18 September 2026. While reading the CSV we found that the agent loop accepted tool results the model had made up (Section VI-A), so these pass rates mostly measure the model's own knowledge rather than tool use. They will be replaced after the corrected benchmark is re-run.

B. Provisional pass rates

Table I and Fig. 3 give the judged pass rates. Both agents failed bench-14, which asked for 50 USD in yen; both answered 7,500 yen and the judge marked it as a wrong rate. ReAct also failed bench-03 (100 USD to euros), where it used all 6 steps without finishing. DFSDT got *Unsure* on bench-03 and bench-10 because the judge could not check a live exchange rate.

C. Why we do not trust these numbers yet

The `react_steps` and `dfsdt_steps` columns are 1 in 37 of the 38 runs. A three-tool question cannot be solved honestly in one step. The answers also disagree with our own mock tools. The `weather` tool returns "Cloudy, 18 °C" for Paris and "Clear, 27 °C" for Tokyo, but the agents reported 15 °C and partly cloudy for Paris (bench-02) and 18 °C and partly cloudy for Tokyo (bench-11). The judge

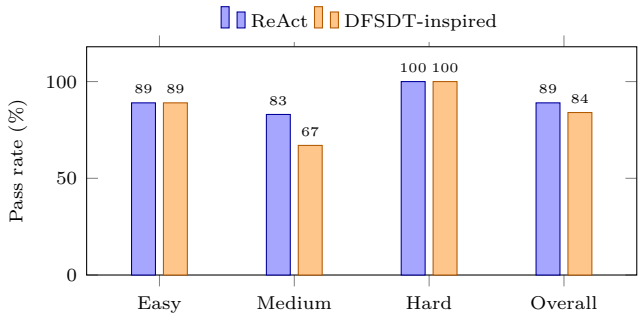


Fig. 3. Provisional pass rate by instruction complexity, redrawn from our own `results/comparison_table.md`.

TABLE II
RETRIEVERS ON THE 19 BENCHMARK INSTRUCTIONS (9 TOOLS)

Retriever	Top-1 correct	All needed in top 3	Top score is 0.0
Keyword (Jaccard)	12	11	2
TF-IDF (default)	12	11	2
MiniLM embeddings	17	17	0

passed all of these because it never saw the tool output. Hard tasks scoring 100% is therefore a sign of the bug, not of skill.

D. Retriever check

Because the benchmark skips retrieval, we measured the retrievers on their own. For each benchmark instruction we ranked the 9 keyless tools and checked the top 3 (Table II). This needs no model calls, so the numbers are final.

The semantic retriever was clearly better. On "Convert 100 US dollars to Euros." it ranked `currency_exchange` first with a score of 0.601, while both word-based retrievers gave every tool 0.0 (Section VI-C).

VI. Challenges and Lessons Learned

A. The loop accepted made-up tool results

Our prompt shows the model the full pattern `Thought / Action / Observation / ... / Finish[...]`, and we call the model with no stop sequence. The model therefore often writes an action, then a fake observation, then `Finish[...]` in one reply. `ReActAgent` checks for `Finish` before it checks for an action, so it returns the final answer and never calls the tool. `DFSDTAgent` does the same. The evidence is in Section V-C. Our unit tests did not catch it because every scripted `EchoLLM` reply contains only one action or one finish.

The fix we have designed is to cut each reply at the first `Observation:` line before parsing, to handle an action before a finish in the same reply, and to tell the model to stop after each action. We also plan a test where one scripted reply contains a fake observation. This fix is not yet merged; its status is [**TO BE FILLED**].

B. Argument parsing

We found three problems in `agent/parsing.py`, all reproduced offline:

1) `ACTION_PATTERN` uses a greedy `(.*)\]` with `DOTALL`, so on a multi-part reply the arguments run to the last `]`. For `calculator[23 * 17]` followed by a fake observation and `Finish[391]`, the captured argument was `23 * 17]\nObservation: 391\nFinish[391`.
2) `parse_args()` splits on every comma. The documented OpenWeatherMap form `city=London,GB` becomes `{'city': 'London', 'GB': ''}`, and positional `USD, EUR, 100` is passed whole as `base`.
3) All values stay strings. A well-formed `currency_exchange[base=USD, target=EUR, amount=100]` always fails, because `CurrencyExchangeAPI.call()` divides a float by `'100'`. With a mocked HTTP response the observation was `Error: unsupported operand type(s) for /: 'float' and 'str'`. This fits ReAct's six-step failure on bench-03, though we did not save that trace and cannot confirm it.

An earlier and related bug is fixed. When a model left out a required argument, `api.call(**kwargs)` raised `TypeError` and the whole run crashed. Commit `495b278` wrapped the call: ReAct now records an `Error:` observation and DFSDT treats the branch as failed. Two tests cover this. The other three problems are still open: [**TO BE FILLED**].

C. Retriever scored 0.0 on unstemmed words

Both word-based retrievers compare exact lower-case tokens. The `currency_exchange` description says "Converts ... currencies", but users write "convert" and "dollars". With no shared token, the keyword and TF-IDF scores were 0.0 for every tool, so the top 3 was just registry order (calculator, weather, search). The same happened for bench-01, "What is 23 times 17?", because the `calculator` description never uses the word "times". Stemming [18] or richer descriptions would help the word-based methods; the MiniLM retriever already avoids the problem (Table II). Making it the default in the UI is still [**TO BE FILLED**].

D. Free-tier rate limits

A benchmark pass makes at least four model calls per instruction (one for each agent and one judge call for each answer), and DFSDT can make up to 3 calls per level. On a free Gemini key this ran into HTTP 429 errors. We added the retry loop in `GeminiLLM` and the friendly 429 message in the UI. We have not yet measured how long a full benchmark run takes: [**TO BE FILLED**].

E. Slow install

Installing `sentence-transformers` also installs PyTorch, which makes the `requirements.txt` install by far the slowest setup step. We kept TF-IDF as the default, and `SentenceTransformerEmbedder` imports the library inside `__init__` so that importing the retriever package stays light. Our finished virtual environment (PyTorch 2.14.0 CPU build, sentence-transformers 6.0.1, Streamlit 1.64.0) takes 1.6 GB on disk. We did not time a clean install: [**TO BE FILLED**].

F. Model-agnostic code paid off

We started on the Anthropic API and later moved the benchmark to Gemini. Because both agents only see `BaseLLM`, the switch was one new 63-line class and one environment variable.

VII. Testing

The `tests/` folder has 52 tests in 7 files, and all 52 pass (`pytest`, about 49 s on our machine). Agent tests use `EchoLLM` to script the model: one checks that ReAct calls the calculator and finishes with 89; another checks that DFSDT backtracks past a failing `weather[city=Atlantis]` call and reports `explored_paths` = 3. The eight web-API wrappers are tested with `unittest.mock.patch` on `requests.get`, so the suite needs no network. The judge tests feed scripted judge replies, including malformed ones, to check verdict parsing. The semantic retriever test skips itself if `sentence-transformers` is missing.

The main gap is the one Section VI-A exposed: our tests assume the model behaves, with one action per reply. No test yet covers multi-part replies, commas inside values, or numeric arguments.

VIII. Limitations

- The benchmark results are provisional for the reason above.
- 19 instructions written by us is a very small test set, and we wrote them knowing which tools exist.
- The judge is the same model as the agents and only sees the final answer, not the tool outputs. Self-preference is a known judge bias [9]. Questions about live exchange rates cannot be checked by the judge at all.
- Three of our tools are mocks with tiny data (three cities, four sentences).
- The benchmark does not use the retriever, so the end-to-end pipeline that the UI runs has not been scored.
- The harness saves only answers and step counts, not full traces.

IX. Future Work

1) Merge the fixes in Sections VI-A–VI-C, add tests for them, and re-run the benchmark; then replace every provisional value.
2) Save the full step trace of every run as JSON next to the CSV.
3) Give the judge the observations, use a different model as judge, and run the win-rate comparison.
4) Show failed siblings to the model in `DFSDTAgent` so retries differ, closer to ToolLLM's DFSDT.
5) Run the benchmark through the retriever with a smaller k, and add instructions where several tools look relevant.

X. Conclusion

We built a working small-scale version of the ToolLLM pipeline: tool wrappers, three retrievers, a ReAct agent, a DFSDT-inspired search agent, an LLM judge and a UI, with 52 passing tests. The most useful thing we learned came from reading our own results: a high pass rate meant little once we saw that the model was writing its own observations. Our first numbers (89% for ReAct, 84% for DFSDT) are therefore provisional. The retriever comparison, which needed no model, gave a firm result: sentence embeddings found a needed tool first in 17 of 19 cases against 12 for TF-IDF. Phase 2 starts with the fixes listed above and a clean re-run.

References

[1] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2307.16789.

[2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2210.03629.

[3] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2305.10601.

[4] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[5] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] P. Jaccard, "The distribution of the flora in the alpine zone," *New Phytologist*, vol. 11, no. 2, pp. 37–50, 1912.

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992, arXiv:1908.10084.

[8] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, arXiv:2002.10957.

[9] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023, arXiv:2306.05685.
[10] "Frankfurter: Foreign exchange rates and currency conversion API," https://www.frankfurter.app, rates published by the European Central Bank.
[11] "Wikipedia REST API: page summary endpoint," https://en.wikipedia.org/api/rest_v1/.
[12] "Free dictionary API," https://dictionaryapi.dev.
[13] "JokeAPI v2," https://v2.jokeapi.dev.
[14] "Quotable: a free, open source quotations API," https://api.quotable.io.
[15] "REST countries," https://restcountries.com.
[16] Google, "Gemini API documentation," https://ai.google.dev/gemini-api/docs.
[17] Streamlit, "Streamlit: A faster way to build and share data apps," https://streamlit.io, Python library; version 1.64.0 used in this project.
[18] M. F. Porter, "An algorithm for suffix stripping," *Program*, vol. 14, no. 3, pp. 130–137, 1980.

Appendix A
Sample Trace

Our harness does not store intermediate steps, so the most detailed record we have is one CSV row. Below is the ReAct row for bench-11 from `results/benchmark_results.csv`, exactly as saved (answer and rationale fields wrapped for width):

```
id:          bench-11  (medium)
instruction: What's the weather in Tokyo, and what is
             the population of Japan?
expected:    weather;country_info
react_steps: 1
answer:      The weather in Tokyo is 18°C and partly
             cloudy, and the population of Japan is
             125,800,000.
verdict:     pass
rationale:   The agent's final answer successfully
             provides both the current weather in Tokyo
             and the population of Japan as requested.
```

For comparison, this is what our `weather` tool actually returns for Tokyo when called directly:

```
>>> WeatherAPI().call(city='Tokyo')
APIResponse(success=True,
  data={'condition': 'Clear', 'temperature_c': 27},
  error=None)
```

The step count of 1 and the mismatched weather show the made-up observation problem in one example. A full Thought / Action / Observation trace from the corrected agent will be added here: [**TO BE FILLED**].